COMPTE RENDU HEBDOMADAIRE

Semaine N°5 — Projet JAEI

8 avril – 14 avril 2026

1. Rappel du contexte

La semaine 5 constitue la phase finale de développement de la plateforme JAEI avant déploiement en production. Elle a couvert la refonte complète du processus de soumission d'article sous forme d'un assistant en 3 étapes, l'intégration de l'intelligence artificielle Gemini 1.5 Flash pour l'analyse automatique des fichiers PDF, la mise en place d'un mode de paiement de démonstration, l'acquisition du nom de domaine officiel jaei-journal.org, la configuration de l'adresse email professionnelle contact@jaei-journal.org, ainsi que la résolution de plusieurs anomalies fonctionnelles et d'interface identifiées lors des tests.

2. Objectifs de la semaine

Les objectifs prévus pour la semaine 5 étaient les suivants :

- Refondre le formulaire de soumission d'article en wizard multi-étapes
- Intégrer l'analyse automatique des articles par intelligence artificielle
- Finaliser l'interface de paiement (mode démonstration et production)
- Corriger les anomalies fonctionnelles du dashboard Reviewer
- Renforcer l'interface d'administration pour les actions éditoriales
- Acquérir le nom de domaine et configurer l'infrastructure email
- Préparer l'environnement de déploiement production

3. Réalisations de la semaine

Toutes les réalisations prévues ont été complétées, avec plusieurs améliorations additionnelles. Voici le détail :

3.1 — Wizard de soumission d'article (3 étapes)

Le formulaire de soumission d'article a été entièrement refondu sous la forme d'un assistant progressif en trois étapes. L'étape 1 collecte les informations générales (titre, domaine de recherche, co-auteurs). L'étape 2 permet l'upload du fichier PDF et déclenche l'analyse IA pour générer automatiquement le résumé et les mots-clés. L'étape 3 présente l'interface de paiement. Un mécanisme de défilement automatique vers le haut de page a été implémenté à chaque changement d'étape via useLayoutEffect, garantissant une expérience de navigation fluide. La soumission en base de données est créée à la transition entre l'étape 2 et l'étape 3, évitant les soumissions orphelines en cas d'abandon lors du paiement.

3.2 — Intégration IA Gemini 1.5 Flash (analyse PDF)

Initialement il était prévu d'intégrer **OpenAI GPT-4o-mini** comme IA. Mais pour des raisons économiques et d'efficacité, nous avons plutôt opté pour **Google Gemini 1.5 Flash**, qui est gratuit et tout aussi efficace.

L'intelligence artificielle a été intégrée pour assister les auteurs lors de la rédaction de leur soumission. Le backend dispose désormais d'un endpoint POST /api/ai/extract-pdf qui reçoit le fichier PDF, en extrait le texte via la bibliothèque pdf-parse, puis transmet le contenu textuel à l'API Google Gemini 1.5 Flash pour générer automatiquement un résumé structuré et une liste de mots-clés pertinents. Les résultats sont pré-remplis dans les champs correspondants du formulaire, que l'auteur peut ensuite modifier librement. L'analyse IA s'effectue en quelques secondes et supporte des articles jusqu'à 8 000 mots.

3.3 — Interface de paiement (mode démonstration)

Un mode de paiement de démonstration a été mis en place pour les besoins de la présentation client avant l'intégration des clés Stripe de production. L'interface affiche un formulaire de carte bancaire fictif (numéro 4242 4242 4242 4242, échéance 12/28, CVC 123) accompagné d'un avertissement « Mode développement — Paiement simulé ». Le bouton de confirmation appelle la route POST /api/payments/dev-confirm qui valide directement la soumission sans traitement financier réel. Cette approche permet de démontrer l'intégralité du workflow éditorial au client sans configuration Stripe active. En production, le composant Stripe (PaymentElement) sera activé automatiquement dès la présence des clés API.

3.4 — Actions éditoriales admin dans la page de détail

Afin d'améliorer l'ergonomie du tableau de bord administrateur, l'ensemble des actions éditoriales a été déplacé directement dans la colonne droite de la page de détail d'une soumission. L'administrateur dispose désormais des boutons suivants sans devoir retourner à la liste : assignation d'un évaluateur (statut submitted), demande de révision (statut under_review), acceptation ou rejet (statut revised), et publication de l'article (statut accepted). Chaque action est conditionnée au statut courant de la soumission et affiche un champ de commentaire éditorial facultatif. La page se met à jour en temps réel après chaque action.

3.5 — Correction du dashboard Reviewer (2 anomalies)

Deux anomalies critiques affectant le dashboard des évaluateurs ont été identifiées et corrigées.

- Anomalie 1 — « Unable to load the article » : le composant ReviewArticle.jsx transmettait l'identifiant de soumission à un endpoint attendant un identifiant de review, provoquant systématiquement une erreur 404. Correction : ajout d'un nouvel endpoint backend GET /api/reviews/by-submission/:submissionId qui retrouve la review du reviewer connecté à partir de l'identifiant de soumission, et retourne conjointement les données de la soumission et l'identifiant de la review.

- Anomalie 2 — Bouton « Review » visible pour les articles publiés : le bouton permettant de soumettre une évaluation restait affiché pour des articles en statut published, accepted ou rejected. Correction : affichage conditionnel limité au seul statut under_review.

3.6 — Corrections d'interface utilisateur

Quatre anomalies d'interface ont été résolues :

- Espace blanc de 56 px dans la sidebar : causé par le positionnement sticky top-14 à l'intérieur d'un conteneur flex déjà positionné sous le header. Suppression des classes conflictuelles.
- Débordement de texte sans espaces (abstract) : les chaînes continues sans espaces (ex. : contenu de test) débordaient du conteneur. Ajout des propriétés CSS wordBreak: break-word et overflowWrap: break-word sur tous les champs concernés.
- Avatars cassés (image non trouvée) : absence de fallback en cas d'URL d'avatar invalide. Ajout d'un gestionnaire onError avec affichage des initiales de l'utilisateur.
- Scroll non réinitialisé entre les étapes du wizard : useEffect insuffisant car exécuté après le rendu. Remplacement par useLayoutEffect pour garantir le défilement synchrone.

3.7 — Acquisition du nom de domaine et configuration email

Le nom de domaine officiel de la plateforme, jaei-journal.org, a été réservé sur la plateforme Infomaniak pour un montant de 9,20 € HT par an. L'adresse email professionnelle contact@jaei-journal.org a simultanément été créée et configurée comme point de contact unique pour l'ensemble des communications de la plateforme (notifications automatiques, support, bureau éditorial). La configuration SMTP (serveur mail.infomaniak.com, port 587) a été intégrée dans les fichiers d'environnement du backend. L'ensemble des occurrences d'anciennes adresses email provisoires dans le code source a été remplacé par contact@jaei-journal.org.

3.8 — Préparation au déploiement production

Un fichier .env.production a été créé et documenté, regroupant l'ensemble des variables d'environnement nécessaires au déploiement sur Render : URL de base de données PostgreSQL, clé JWT sécurisée, identifiants SMTP Infomaniak, clés Cloudinary, clé API Gemini et URL de frontend en production (https://jaei-journal.org). Ce fichier est exclu du dépôt Git via .gitignore. La progression globale du projet a été mise à jour dans le fichier docs/PROGRESSION.md.

4. Difficultés rencontrées et solutions

4.1 — Incompatibilité pdf-parse v2.x

Cause : la version 2.x de la bibliothèque pdf-parse tente, lors de son chargement via require('pdf-parse'), de lire un fichier de test dont le chemin absolu est invalide dans l'environnement d'exécution Node.js, provoquant une erreur immédiate.

Solution : import direct via le chemin interne require('pdf-parse/lib/pdf-parse.js'), contournant le point d'entrée principal défaillant. L'extraction de texte fonctionne correctement.

4.2 — API Gemini indisponible en version v1beta pour le multimodal

Cause : le modèle gemini-1.5-flash n'est pas disponible pour les requêtes multimodales (envoi de PDF encodé en base64) via la version v1beta de l'API Google Generative AI.

Solution : extraction préalable du texte du PDF avec pdf-parse, puis transmission du contenu textuel à Gemini en tant que simple chaîne de caractères. Cette approche est compatible avec toutes les versions de l'API et produit des résultats de qualité équivalente.

4.3 — Corps de requête undefined dans les routes de paiement

Cause : le middleware express.json() était monté dans server.js après la déclaration de la route /api/payments, rendant req.body systématiquement undefined pour toutes les requêtes de paiement.

Solution : déplacement du middleware express.json() avant le montage des routes de paiement dans server.js. Les requêtes POST reçoivent désormais correctement leur corps JSON.

5. État d'avancement global

Le tableau ci-dessous présente l'état d'avancement de l'ensemble du projet à la date du rapport :

Phase	Contenu	Statut
Semaine 1	Infrastructure, authentification, base de données	☑ **Terminé**
Semaine 2	Dashboards, soumissions, uploads, emails	☑ **Terminé**
Semaine 3	Site public, catalogue, pages légales	☑ **Terminé**
Semaine 4	IA (OpenAI), Stripe backend, corrections bugs	☑ **Terminé**
Semaine 5	Wizard soumission, IA Gemini, domaine, email, corrections	☑ **Terminé**
Semaine 6	Déploiement Render, DNS, Stripe prod, tests, livraison	⏳ **En cours**

6. Prévisions pour la semaine 6 (livraison finale)

La semaine 6 constitue la dernière phase du projet et aboutira à la livraison finale de la plateforme au client. Les travaux porteront sur :

1. Déploiement de la base de données PostgreSQL sur Render
2. Déploiement du backend Node.js sur Render avec configuration des variables d'environnement de production
3. Déploiement du frontend React sur Render
4. Pointage du nom de domaine jaei-journal.org vers les services Render (configuration DNS Infomaniak)
5. Intégration des clés Stripe
6. Tests end-to-end complets de la chaîne éditoriale complète en environnement de production
7. Livraison finale et remise des accès au client